

PAPER DRAFT · EBPf SECURITY

eBPF 기반 AI 서빙 프레임워크 취약점 가상 패치(Virtual Patching) 프레임워크 kShield- VirtualPatch

이상헌 · lsh66404865@gmail.com

- 작성 상태 (v3 + LSM + v4/v5/v6/v7).** SHADOW_EXEC에 더해 실행 파일 블록리스트 우회(bash `/dev/tcp/`)를 막는 SHADOW_CONNECT를 추가하고 VM에서 검증하였다. 이후 LSM 혹은 기반 동기적 사전 차단(3.5절)을 추가로 구현하고 기능·성능 검증(AI 워커 계보와 무관한 프로세스에 대한 영향 포함)까지 VM에서 완료하였다 — 이 과정에서 v3에도 있던 실행 파일 블록리스트 누락 버그를 함께 발견·수정하였다. fork 집약적 워크로드에서는 처음으로 통계적으로 유의미하지만 작은 오버헤드(처리량 -0.67%, 지연시간 +0.7%)를 관측하였다. 이후 curl/wget을 실행 즉시 차단 목록에서 제외하고 목적지 인지 계층에 판단을 넘긴 v4, 재컴파일 없는 audit-only 모드를 추가한 v5(3.6절)까지 VM에서 검증하였다. 제출 전 자체 검토 과정에서 계보 판정이 "자손 프로세스"만 커버하고 감시 대상 프로세스 자신의 직접 행위는 놓치는 구조적 공백을 발견하여 `watched_self[]` 기반 자기 자신 검사를 추가한 v6(3.7절)까지 VM에서 검증하였다. 이후 기획 관점 재검토에서 데몬이 클러스터보다 나중에 뜨거나 재시작되면 계보 정보가 유실되는 문제를 추가로 발견하여, 유저스페이스 로더가 기동 시 `/proc` 을 스캔해 계보를 백필하도록 수정하였고 (3.8절, v7), 직속 부모 검사로는 잡을 수 없는 2단계 손자뻘 체인으로 VM 재현·검증을 완료하였다. 참고문헌은 웹 검색으로 실제 URL·발행일자를 확인하여 정리하였다.

요약 (ABSTRACT)

AI 서빙 프레임워크(Ray, vLLM, Triton 등)에서 발견되는 원격 코드 실행(RCE) 취약점은 벤더의 패치 대응이 지연되거나, 조직 내부적으로 프레임워크 업그레이드가 늦어지는 경우가 많다. 대표적으로 Ray의 Jobs Submission API에 존재하는 인증 부재 취약점(CVE-2023-48022, "ShadowRay")은 벤더가 공식 패치를 제공하지 않은 채 오랜 기간 방치되었고, 실제로 수천 대의 클러스터가 침해되어 암호화폐 채굴 등에 악용되었다. 본 논문은 eBPF를 활용하여 애플리케이션 코드를 수정하거나 재배포하지 않고도, 이러한 취약점이 실제로 악용될 때 나타나는 비정상 행위를 커널 수준에서 탐지·차단하는 가상 패치(virtual patching) 메커니즘 kShield-VirtualPatch를 제안한다. 감시 대상 AI 워커 프로세스의 계보(lineage)를 추적하여, 의심스러운 자식 프로세스 실행(SHADOW_EXEC)뿐 아니라 바이너리 종류와 무관하게 신뢰되지 않은 목적지로의 아웃바운드 연결 시도(SHADOW_CONNECT)를 "행위의 본질적 불변량" 기준으로 탐지한다 — 이는 "어떤 도구를 실행했는가"가 아니라 "무엇을 하려 했는가"를 감시하므로, bash 내장 기능처럼 별도 바이너리를 실행하지 않

는 우회에도 대응한다. 아울러 비동기 `bpf_send_signal` SIGKILL의 잔여 한계를 보완하기 위해, LSM 혹은 (`security_bprm_check_security` / `security_socket_connect`)으로 시스템 콜 자체를 동기적으로 실패시키는 사전 차단 컴포넌트도 함께 구현하였다.

mock Ray Jobs API 환경에서 다단계 공격 체인(`python3`→`sh`→`curl`)과 `bash /dev/tcp/` 우회 시도를 각각 실행·연결 시점에 정확히 차단하면서 정상 job은 오탐 없이 통과함을 VM 실측으로 확인하였다. 비활성/SHADOW_EXEC+CONNECT/LSM 사전 차단 3개 그룹을 10회씩 반복 측정한 결과, fork 단계가 얇은 워크로드와 계보 무관 프로세스의 `connect()` 성능 모두에서 통계적으로 유의미한 차이는 없었다(Welch's t-test). 다만 job당 fork를 50배 늘린 워크로드에서는 처음으로 유의미하지만 작은 차이(처리량 -0.67%, 지연 시간 +0.7%)가 나타나, 오버헤드가 워크로드의 fork 빈도에 비례함을 확인하였다. 개발 및 제출 전 검토 과정에서 실행 파일 블록리스트의 경로-별칭 누락, 이중 용도 도구(`curl`/`wget`)의 오탐 전제, 감시 대상 프로세스 자신의 직접 행위 탐지 공백, 데몬 재시작 시 계보 유실 등 설계 자체의 구조적 공백을 추가로 발견하여 재설계·검증하였다(3.3, 3.6~3.8절) — 이 반복 과정 자체가 행위 기반 탐지 메커니즘을 설계할 때 반드시 고려해야 할 원칙들을 보여준다(5장).

eBPF

가상 패치

virtual patching

AI 서빙 프레임워크 보안

Ray

ShadowRay

CVE-2023-48022

1 서론

AI 서빙 프레임워크(Ray, vLLM, Triton Inference Server, MLflow 등)는 대규모 LLM 추론 파이프라인의 핵심 인프라로 자리잡았다. 그러나 이러한 프레임워크들은 빠른 기능 개발 속도에 비해 보안 검토가 상대적으로 미흡한 경우가 많으며, 실제로 원격 코드 실행급 심각한 취약점이 지속적으로 보고되고 있다.

가장 널리 알려진 사례는 Ray의 Jobs Submission API에 존재하는 CVE-2023-48022("ShadowRay")이다. Ray 대시보드의 기본 포트(8265)로 노출되는 이 API는 기본 설정에서 별도의 인증을 요구하지 않아, 네트워크로 접근 가능한 누구나 임의의 코드를 클러스터의 워커 프로세스 권한으로 실행시킬 수 있다. 보안 업체 Oligo Security의 조사에 따르면¹, 이 취약점을 통해 실제로 수천 대의 Ray 클러스터가 침해되어 암호화폐 채굴, 역방향 셸 연결, 클러스터 내 크레덴셜 탈취 등에 악용된 사례가 확인되었다. 더 심각한 문제는 벤더(Anyscale)가 이를 "설계상 의도된 동작(Ray는 신뢰된 네트워크 내에서만 배포되어야 한다는 문서상의 전제)"이라는 입장을 오래 유지하며 즉각적인 공식 패치를 제공하지 않았다는 점이다². 이는 전통적인 "벤더 패치를 기다린다"는 대응 전략이 통하지 않는 상황을 만든다.

이러한 상황에서 방어자가 취할 수 있는 현실적인 선택지는 제한적이다. 프레임워크 자체를 수정하는 것은 벤더의 협조 없이는 어렵고, 프레임워크 업그레이드는 프로덕션 환경의 호환성 리스크 때문에 신중하게 이루어져야

한다. 웹 애플리케이션 보안 분야에서는 이런 상황에 대응하기 위해 WAF(Web Application Firewall) 수준에서 "가상 패치(virtual patching)"라는 개념이 오래전부터 활용되어 왔다⁷ — 취약점 자체를 고치지 못하는 동안, 그 취약점이 악용될 때 나타나는 특정 패턴만 탐지하여 임시로 차단하는 기법이다.

본 논문은 이 개념을 AI 서빙 프레임워크의 커널 수준으로 확장한다. eBPF를 이용하면 애플리케이션 코드를 한 줄도 수정하지 않고, 재시작도 없이, 실행 중인 프로세스에 탐지 로직을 "붙였다 떼어낼 수 있다." 이는 정식 패치가 배포되기 전까지의 공백을 메우는 임시 방어선이자, 정식 패치가 나온 뒤에는 혹을 제거하기만 하면 되는 저비용 대응 수단이 된다.

본 연구의 주요 기여는 다음과 같다.

- AI 서빙 프레임워크의 알려진 RCE 취약점 중 ShadowRay를 사례 연구로 삼아 eBPF 기반 가상 패치 메커니즘을 설계·구현 — 다른 프레임워크·CVE로의 일반화는 검증하지 않았으며 향후 연구로 남긴다(5장)
- 파일 접근 "빈도"가 아닌 "비정상 행위 발생 여부"를 탐지 기준으로 삼아, 단일 악용 시도로도 즉시 탐지 가능한 구조 제시
- 취약점 악용 재현(mock Ray Jobs API + exploit 스크립트)과 eBPF 방어 효과의 실증적 검증

2 관련 연구

2.1 AI 서빙 프레임워크의 보안 취약점

Ray의 ShadowRay(CVE-2023-48022^{3 4}) 외에도, TorchServe, Triton Inference Server, MLflow 등 주요 AI 서빙 프레임워크에서 경로 탐색(path traversal), 역직렬화(deserialization) 기반 원격 코드 실행 등의 취약점이 다수 보고되었다. 이들 프레임워크는 공통적으로 (1) 빠른 기능 릴리스 주기로 인해 보안 검토가 상대적으로 후순위로 밀리고, (2) 조직 내부적으로 버전 업그레이드가 프로덕션 안정성 문제로 지연되는 경향을 보인다.

2.2 가상 패치(Virtual Patching)

가상 패치는 웹 애플리케이션 보안 분야에서 오래전부터 확립된 개념으로, ModSecurity, Cloudflare WAF, AWS WAF 등이 신규 CVE 발표 시 해당 취약점의 악용 패턴을 차단하는 임시 규칙을 신속히 배포하는 방식으로 널리 활용한다. 그러나 이러한 도구들은 대부분 HTTP 요청 계층에서 동작하며, 애플리케이션이 실제로 수행하는 시스템 수준 행위(프로세스 실행, 파일 접근 등)까지는 관측하지 못한다.

2.3 eBPF 기반 런타임 보안 및 가상 패치 메커니즘

Falco⁵, Tetragon⁶ 등 eBPF 기반 런타임 보안 도구는 커스텀 룰을 통해 특정 시스템 콜 패턴을 탐지·차단하는 기능을 이미 범용적으로 제공한다. 즉, "eBPF로 특정 행위 패턴을 감시해서 막는다"는 메커니즘 자체는 새로운 것이 아니다. 그러나 이들 도구의 기본 룰셋은 범용 워크로드를 대상으로 하며, AI 서빙 프레임워크에 특화된 취약점(예: ShadowRay의 구체적인 악용 시 발생하는 프로세스 트리 패턴)에 대한 룰은 아직 정립되어 있지 않다. 본 논문의 기여는 새로운 탐지 메커니즘의 발명이 아니라, 기존 eBPF 런타임 보안 기법을 AI 서빙 프레임워크라는 특정 도메인에 적용하고 실제 CVE로 그 효과를 검증하는 데 있다.

기존 도구 위에 룰을 엮는 대신 별도의 최소 구현을 처음부터 작성한 것은, 룰 갱신·알림 연동 같은 프로덕션 편의 기능을 갖추는 것보다 "AI 워커 계보 추적 + exec/connect 이중 계층 + 동기적 사전 차단"이라는 탐지 메커니즘 자체를, 연구 목적에 맞게 외부 의존성 없이 정확히 통제된 형태로 구현·측정하는 것을 우선했기 때문이다. 이는 실용적 배포 도구로서는 Falco/Tetragon 대비 열위인 지점(재컴파일 없는 룰 갱신, SIEM 연동, 다중 커널 버전 검증 등은 향후 연구로 남김, 5장)이며, 두 접근을 정량적으로 비교하는 것 자체도 아직 수행하지 못한 과제다(5장).

2.4 kShield(모델 파일 보안)와의 관계

본 연구는 저자의 선행 연구인 kShield(모델 파일 보안 프레임워크)⁸와 같은 eBPF kprobe/tracepoint 기반 아키텍처(커널 공간 BPF 프로그램 + perf buffer + 사용자 공간 데몬 + `bpf_send_signal` 을 통한 즉시 프로세스 종료)를 공유한다. 다만 탐지 대상과 원리가 다르다. kShield는 보안 파일에 대한 접근 빈도(임계값 초과)를 탐지 기준으로 삼는 반면, 본 연구는 정상적으로는 절대 발생하지 않을 **단일 행위**(비정상 프로세스 실행)의 발생 여부를 탐지 기준으로 삼는다. 이는 공격자가 파일을 여러 번 나누어 여는 대신 한 번의 `open()` + `read()` 로 데이터를 통째로 탈취하는 경우에도 탐지가 가능하다는 구조적 이점을 가진다.

3 설계 및 구현

본 절 전체(3.3, 3.5~3.8절)에 걸쳐 설계가 v1부터 v7까지 총 여섯 차례 반복되었다. 각 반복은 예외 없이 VM 실측이나 제출 전 자체 검토 과정에서 발견된 구체적 문제에 대한 대응이었으며, [표 1]은 이를 한눈에 볼 수 있도록 요약한 것이다 — 각 행의 상세 서술은 마지막 열의 절을 참고한다.

[표 1] 설계 반복(v1~v7) 요약

버전	발견한 문제	수정 내용	검증 방법	절
----	--------	-------	-------	---

v1	직속 부모만 확인 → 다단계 공격 체인을 놓치고 정상 job도 오탐	프로세스 계보(lineage) 추적 도입	VM 실측	3.3
v2	실행 파일 이름 블록리스트 (<code>suspicious_bins[]</code>)가 bash 내장 기능(<code>/dev/tcp/</code>) 등 우회에 취약	연결 시도 자체를 감시하는 SHADOW_CONNECT 추가	VM 실측	3.3
v3	블록리스트가 <code>/usr/bin/curl</code> 은 막고 <code>/bin/curl</code> 은 놓침(경로 별칭 누락); 비동기 SIGKILL의 사후 차단 한계	별칭 경로 전부 등록 + LSM 혹 기반 동기적 사전 차단(<code>-EPERM</code>) 컴포넌트 추가	VM 실측(빌드·attach·기능·성능)	3.3, 3.5
v4	curl/wget을 목적지와 무관하게 즉시 차단 → 정당한 모델·데이터셋 다운로드까지 오탐	curl/wget을 exec 블록리스트에서 제외, 판단을 SHADOW_CONNECT/LSM <code>socket_connect</code> 로 이전	신뢰/비신뢰 목적지 양쪽 VM 실측	3.6
v5	신규 룰을 처음부터 즉시 차단으로만 배포(단계적 롤아웃 불가)	재컴파일 없이 전환 가능한 <code>enforce_mode</code> (audit-only/enforce) 런타임 토글 추가	v3·LSM 양쪽, enforce/audit-only 4가지 조합 VM 실측	3.6
v6	계보 판정이 "자손"만 커버 → 감시 대상 프로세스 자신의 fork 없는 직접 행위는 어느 혹도 못 잡음	자기 자신의 comm을 확인하는 <code>watched_self[]</code> 추가	이름 바꾼 프로세스 직접 실행 + 무관 프로세스 회귀 확인, VM 실측	3.7
v7	계보 맵이 fork 혹에만 의존 → 데몬이 클러스터보다 나중에 뜨거나 재시작되면 이미 떠 있던 자손 프로세스가 계보에서 누락	유저스페이스 로더가 기동 시 <code>/proc</code> 을 스캔해 계보를 백필	직속 부모 검사로 못 잡는 손자뻘(2단계) 체인으로 v3·LSM 양쪽 VM 실측	3.8

3.1 위협 모델

본 논문이 상정하는 위협 모델은 다음과 같다. 공격자는 AI 서빙 프레임워크의 관리 API(예: Ray Jobs Submission API, 기본 포트 8265)에 네트워크로 접근 가능하며, 해당 API에 인증 메커니즘이 부재하거나 미흡하게 설정된 취약점(CVE-2023-48022^{3 4})을 악용하여 임의의 코드를 원격에서 실행시킬 수 있다.

이 취약점은 벤더가 "의도된 설계"라는 입장을 고수하며 즉각적인 공식 패치를 제공하지 않거나², 조직 내부적으로 프레임워크 업그레이드에 따른 프로덕션 호환성 리스크 때문에 패치 적용이 장기간 지연되는 상황을 전제로 한다. 실제로 Oligo Security의 조사에 따르면¹ 수천 대의 Ray 클러스터가 이 취약점을 통해 실제로 침해되어 암호화폐 채굴, 역방향 셸 연결, 클러스터 내 자격 증명 탈취 등에 악용된 사례가 확인되었다.

공격자의 궁극적 목표는 원격 코드 실행을 통해 정상적인 AI 워커 프로세스의 권한으로 비정상적인 하위 프로세스를 실행하거나(예: `/bin/sh`, `curl`), 공격자가 제어하는 외부 서버로 아웃바운드 연결을 수립하여 2차 피해(암호화폐 채굴, 데이터 유출, 크레덴셜 탈취)를 유발하는 것이다.

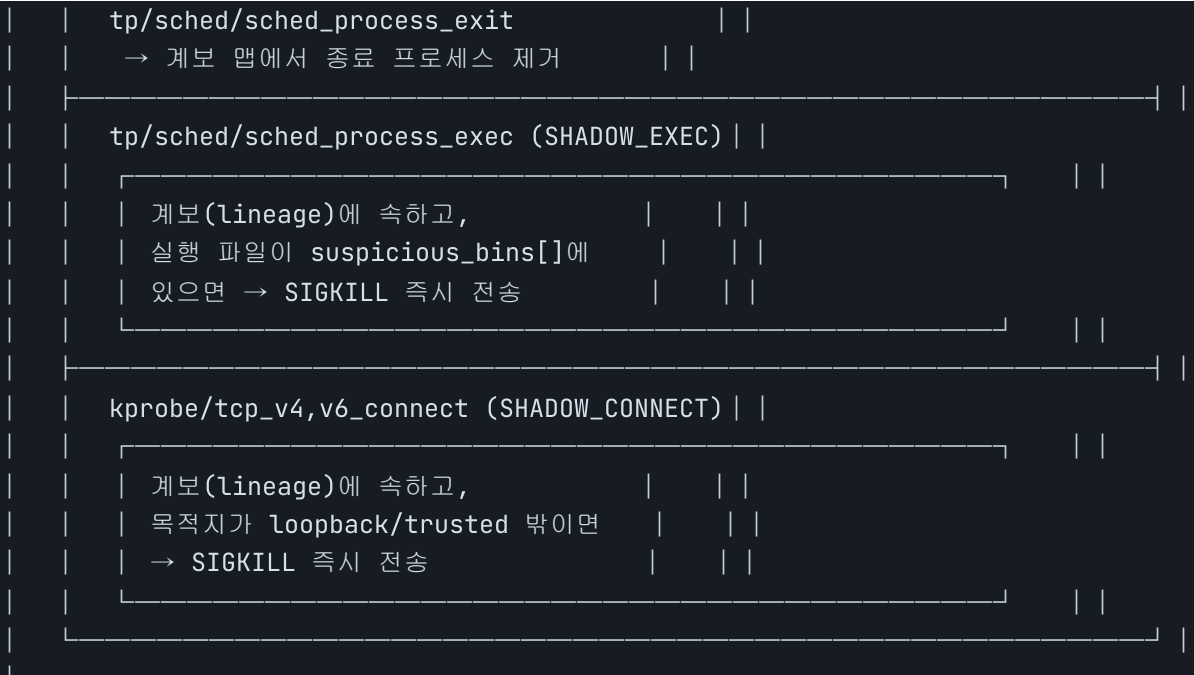
단, 본 논문은 ShadowRay 취약점의 원격 코드 실행 이후 악성 행위 탐지에 집중하며, 취약점 자체의 존재 여부 진단이나 초기 침투 벡터 차단은 다루지 않는다. 또한 프레임워크 소스 코드 자체를 사전에 변조하거나 커널 루트킷을 삽입하는 공격, 그리고 ShadowRay 외 다른 CVE에 대한 일반화된 방어는 본 논문의 범위 밖으로 한다.

윤리적 고려: CVE-2023-48022는 2023년에 이미 공개되었고, 벤더가 "의도된 설계"라는 입장을 유지하며 별도 패치를 제공하지 않고 있어(위 [2]), 본 논문이 새로운 취약점을 폭로하는 것이 아니라 이미 공개되고 실제로 악용되고 있는 취약점에 대한 방어 기법을 다룬다. 모든 공격 재현은 실제 Ray 클러스터가 아닌 저자가 직접 구현한 목업 서버(3.4절)만을 대상으로 하였으며, 공개 저장소의 공격 스크립트 (`attack/exploit_shadowray.py`) 역시 기본 대상이 이 목업 서버로 설정되어 있어 실제 피해 없이 방어 효과만 재현할 수 있도록 하였다.

3.2 시스템 아키텍처

kShield-VirtualPatch는 저자의 선행 연구 kShield⁸와 동일한 아키텍처 패턴(커널 공간 BPF 프로그램 + perf buffer + 사용자 공간 로거)을 따른다.





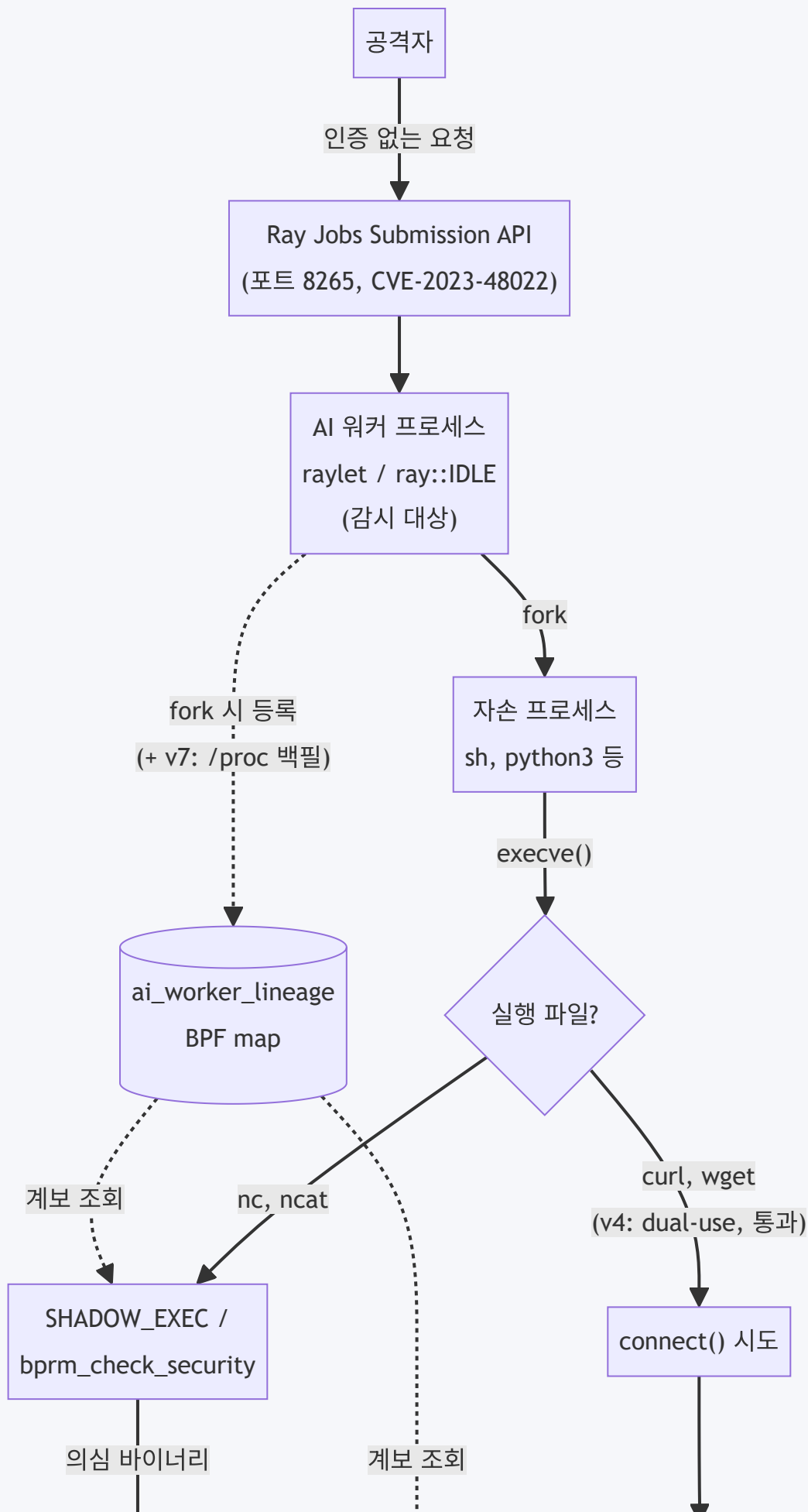
이 다이어그램은 어디서나(추가 커널 설정 없이) attach 가능한 기본 방어선(kprobe/tracepoint + 비동기 SIGKILL)만을 나타낸다. 3.5절의 실험적 LSM 컴포넌트(`kshield_vpatch_lsm`)는 동일한 판정 로직을

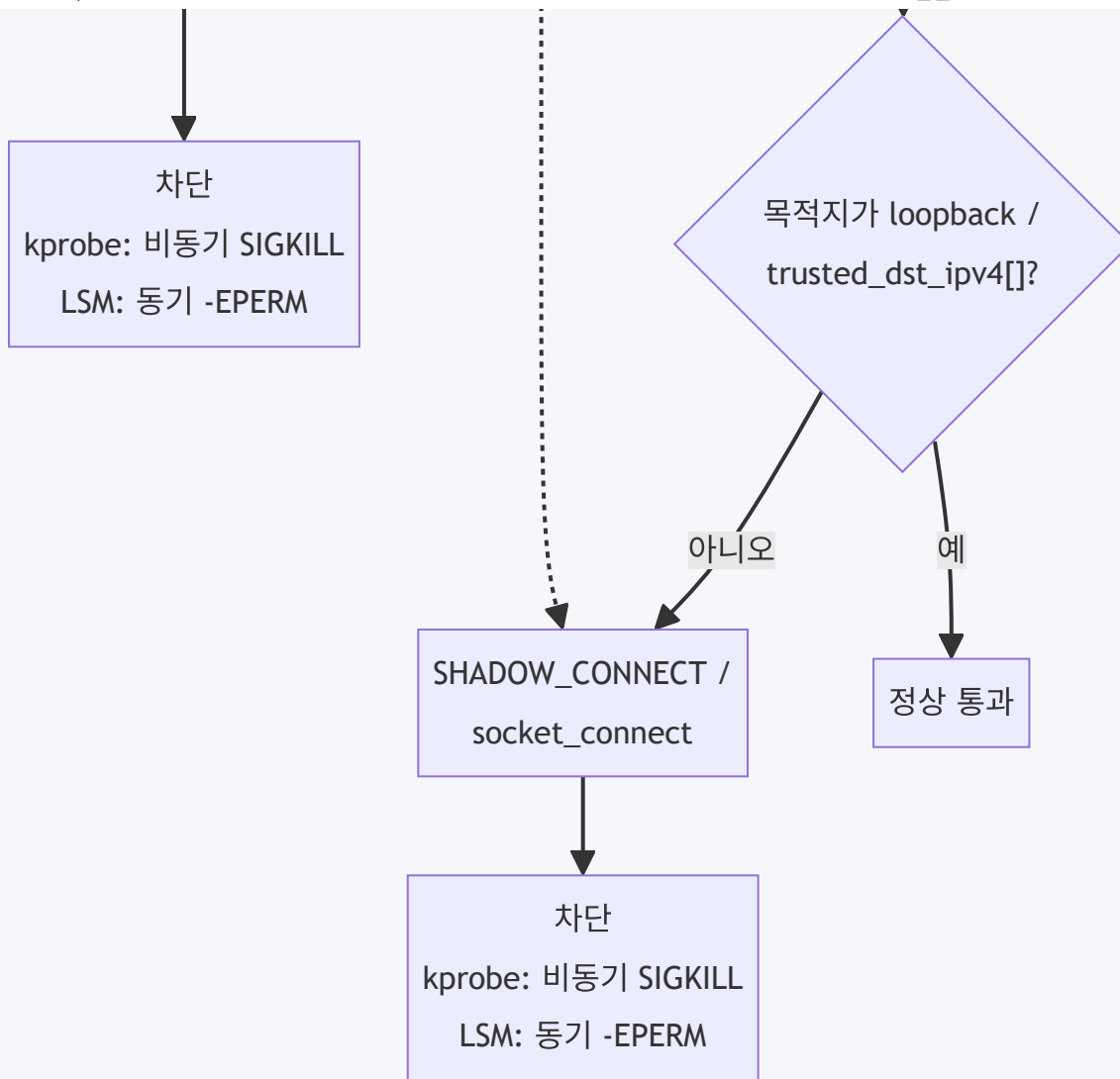
`security_bprm_check_security` / `security_socket_connect` hook에서 동기적으로 수행하는 독립된 BPF 오브젝트이며, 위 다이어그램에는 포함되지 않는다.

3.3 핵심 탐지 메커니즘 (SHADOW_EXEC, SHADOW_CONNECT)

아래 [그림 1]은 ShadowRay 공격 체인과 그 위에 놓인 탐지 지점을 현재(v7 기준) 설계로 요약한 것이다. 세부 판정 로직(v1~v3의 재설계 과정)은 이어지는 본문에서 다룬다.

[그림 1] 공격 체인과 탐지 지점





감시 대상 프로세스 자신이 fork 없이 직접 실행/연결하는 경우(v6, `watched_self[]`)는 위 그림의 "AI 워커 프로세스" 노드에서 fork 단계를 거치지 않고 바로 오른쪽 경로(`execve()` / `connect()`)로 진입하는 것으로 이해하면 된다 — 별도 노드로 그리지 않은 이유는 판정 로직상 두 경로가 이후 완전히 동일하기 때문이다.

초기 설계(v1)는 `sched_process_exec` 트레이스포인트에서 새로 실행된 프로세스의 **직속 부모**만 확인하여, 부모 comm이 `watched_parents[]` (감시 대상 AI 워커 프로세스 목록, 예: `raylet`)와 일치하고 실행 파일이 `suspicious_bins[]` 와 일치하면 즉시 SIGKILL을 전송하는 방식이었다.

그러나 VM 실측 과정에서 두 가지 문제가 발견되었다.

1. **정상 job도 오탐**: mock Ray Jobs API는 정상/악성 job을 구분하지 않고 항상 `/bin/sh -c <entrypoint>` 형태로 실행한다. 이는 실제 Ray의 job 실행 방식과도 동일하다. 따라서 `suspicious_bins[]` 에 `/bin/sh` 가 포함되어 있으면 정상 job(`echo`, `python3 -c ...`)조차 실행 즉시 차단되는 문제가 실측으로 확인되었다.

2. **다단계 실행 체인 탐지 실패**: 실제 공격은 `python3(워커) → sh → curl` 처럼 최소 2단계를 거친다. curl 의 직속 부모는 `sh` 이지 워커 프로세스가 아니므로, "직속 부모만 확인"하는 v1 로직으로는 curl 실행을 탐지할 수 없었다.

이를 해결하기 위해 v2는 `sched_process_fork` 트레이스포인트를 추가로 후킹하여 **AI 워커의 자손 프로세스 계보(lineage)**를 BPF 해시맵(`ai_worker_lineage`, `pid → flag`)으로 추적한다. 프로세스가 fork될 때, 부모가 이미 계보에 속하거나 `watched_parents[]` 와 일치하면 자식도 계보에 편입시킨다. 이렇게 하면 몇 단계를 거치든 계보 추적이 끊기지 않는다. 아울러 `/bin/sh`, `/bin/bash` 는 정상 job 실행에도 쓰이는 경로이므로 `suspicious_bins[]` 에서 제외하고, `curl`, `wget`, `nc` 등 실제 페이로드 다운로드·외부 연결에 쓰이는 바이너리만 남겼다.

최종 탐지 로직(`src/kshield_vpatch.bpf.c` 참조)은 다음과 같다.

1. `sched_process_fork` 시 부모가 계보에 속하거나 `watched_parents[]` 와 일치하면 자식 PID를 계보에 등록
2. `sched_process_exec` 시 현재 프로세스가 계보에 속하는지(또는 직속 부모가 `watched_parents[]` 와 일치하는지) 확인
3. 계보에 속하는 경우, 실행된 바이너리가 `suspicious_bins[]` (`/usr/bin/curl`, `/usr/bin/wget`, `/bin/nc`, `/usr/bin/nc`, `/usr/bin/ncat`)와 일치하는지 확인
4. 일치하면 이벤트를 perf buffer로 제출하고 `bpf_send_signal(SIGKILL)` 호출

이 메커니즘은 kShield의 EVIL_OPEN(반복 횟수 기반)과 근본적으로 다른 원리로 동작한다. AI 워커 계보에서 curl/wget 등이 실행되는 행위는 정상 운영 중에는 **단 한 번도 발생해서는 안 되는 행위**이므로, 반복 횟수와 무관하게 첫 발생 시점에 즉시 탐지·차단할 수 있다.

`watched_parents[]` 와 `suspicious_bins[]` 는 BPF rodata 섹션에 컴파일 타임에 삽입되며, 이는 kShield⁸와 동일하게 대상 경로 변경 시 재컴파일이 필요하다는 한계를 가진다(향후 연구 참고). 전체 구현 코드는 공개 저장소⁹에서 확인할 수 있다.

V3 재 설계 - SHADOW_CONNECT

v2의 SHADOW_EXEC는 `suspicious_bins[]` 라는 실행 파일 이름 블록리스트에 의존하는데, 다음과 같이 쉽게 우회 가능하다는 한계가 있었다.

1. **bash 내장 기능 우회:** `bash -c 'exec 3<>/dev/tcp/host/port; ...'` 방식은 bash 자체의 내장 TCP 리다이렉션 기능을 사용하므로, curl/nc 같은 별도 바이너리를 `exec()` 하지 않는다.
2. **watched_parents의 python3가 suspicious_bins에는 없음:** python3 자신이 소켓 API로 직접 통신하면 실행 파일 이름 기반 검사를 통과한다.
3. **별도 프로세스 실행 없이 직접 통신:** 원래 프로세스 안에서 소켓만 열면, 애초에 감시할 `exec()` 이벤트 자체가 없다.

이 세 우회의 공통점은 결국 커널의 `tcp_v4_connect` / `tcp_v6_connect` 를 거쳐 아웃바운드 TCP 연결을 시도한다는 점이다. v3는 이 지점을 직접 후킹하여, "어떤 바이너리를 실행했는가"가 아니라 **"AI 워커 계보에서 신뢰되지 않은 목적지로 연결을 시도했는가"**를 감시한다. AI 워커 계보에 속한 프로세스가 loopback(127.0.0.0/8) 또는 `trusted_dst_ipv4[]` (운영자가 사전에 등록한 신뢰 목적지 목록)에 없는 주소로 연결을 시도하면, 바이너리 종류와 무관하게 즉시 SIGKILL을 전송한다. 기존 SHADOW_EXEC은 그대로 유지하여 두 계층이 함께 방어한다(defense-in-depth) — 알려진 바이너리는 실행 시점에 더 일찍 잡고, 그 외 모든 경로는 연결 시도 시점에 잡는다.

아울러 v2에는 프로세스 종료 시 `ai_worker_lineage` 맵을 정리하는 로직이 없어, PID가 재사용될 경우 무관한 새 프로세스가 죽은 프로세스의 계보 정보를 잘못 물려받을 수 있는 버그가 있었다. v3는 `sched_process_exit` 후 프로세스 종료 시 계보 정보를 제거하여 이를 해결하였다.

V3의 한계

SHADOW_CONNECT도 완전한 차단은 아니다. (1) 이미 열려 있는 정상 연결에 얹혀 데이터를 빼가는 경우, (2) 허용된 포트/프로토콜(예: DNS) 위로 데이터를 숨기는 터널링, (3) 네트워크가 아예 필요 없는 로컬 전용 공격은 포착되지 않는다. 또한 `bpf_send_signal` 은 비동기적이므로 `connect()` 진입 시점에 신호를 보내도 완전한 사전 차단을 수학적으로 보장하지는 않는다 — 이 한계를 해소하기 위해

`security_socket_connect` / `security_bprm_check_security` LSM 혹 기반의 동기적 사전 차단을 별도 컴포넌트로 구현하였다(3.5절). 다만 `CONFIG_BPF_LSM` 및 활성 LSM 스택에 "bpf" 포함이 필요해 배포 환경 의존성이 있으므로, 본 절의 SHADOW_EXEC/SHADOW_CONNECT는 그러한 의존성 없이 어디서나 동작하는 기본 방어선으로 계속 유지한다.

3.4 공격 재현 환경

실제 Ray 클러스터를 설치하지 않고도 ShadowRay의 핵심 취약점(인증 없는 원격 코드 실행)을 재현하기 위해 mock Jobs API 서버(`attack/mock_ray_server.py`)를 구현하였다. 이 서버는 `/api/jobs/` 엔드포인트로 제출된 `entrypoint` 명령어를 인증 검사 없이 그대로 서브프로세스로 실행하여, 실제 Ray Jobs API의 핵

심 취약 동작을 그대로 재현한다. `attack/exploit_shadowray.py` 는 이 엔드포인트에 악성 명령어를 제출하는 공격 스크립트다.

3.5 LSM 혹은 기반 동기적 사전 차단

SHADOW_EXEC/SHADOW_CONNECT는 모두 "행위가 발생한 뒤 `bpf_send_signal(9)` 로 프로세스를 죽이는" 방식이다. `bpf_send_signal` 은 비동기 전달이므로, 신호가 실제로 전달되기 전에 `connect()` 가 이미 진행되었거나 `execve()` 가 이미 완료되었을 여지를 이론적으로 완전히 배제하지 못한다. 이를 근본적으로 해소하려면 시스템 콜의 반환값 자체를 커널이 원래 진행을 이어가기 전에 대체해야 한다.

`BPF_PROG_TYPE_LSM` 프로그램은 커널의 LSM 혹은 지점(`security_bprm_check_security`, `security_socket_connect` 등)에 `fmod_ret` 트랩폴린 방식으로 부착되어, 원래 함수가 반환하기 전에 그 반환값 자체를 대체할 수 있다. 이를 이용해 별도 컴포넌트(`src/kshield_vpatch_lsm.bpf.c`, `src/kshield_vpatch_lsm.c`)를 구현하였다.

- `security_bprm_check_security` 혹에서 AI 워커 계보가 `suspicious_bins[]` 를 실행하려 하면 `- EPERM` 을 반환하여 `execve()` 자체를 실패시킨다 — 의심 바이너리는 실행되지 않는다.
- `security_socket_connect` 혹에서 AI 워커 계보가 신뢰되지 않은 목적지로 `connect()` 를 시도하면 `- EPERM` 을 반환하여 연결 자체를 실패시킨다 — 3-way handshake조차 시작되지 않는다.

두 혹 모두 커널 호출 경로상 SHADOW_EXEC/SHADOW_CONNECT가 관측하는 지점(`sched_process_exec` tracepoint, `tcp_v4_connect` / `tcp_v6_connect`)보다 앞서 실행되므로, 이 LSM 컴포넌트가 성공적으로 attach된 환경에서는 대부분의 시도가 SHADOW_EXEC/SHADOW_CONNECT까지 도달하기 전에 이미 차단된다. 두 컴포넌트를 동시에 로드해도 서로 간섭하지 않으며, LSM attach가 불가능한 환경에서는 SHADOW_EXEC/SHADOW_CONNECT가 계속 방어선 역할을 한다.

`BPF_PROG_TYPE_LSM` 은 `CONFIG_BPF_LSM=y` 커널과, 활성 LSM 목록(`/sys/kernel/security/lsm`)에 `"bpf"` 포함을 요구한다. 실험 환경(4.1절)에서는 재부팅 전 `/sys/kernel/security/lsm` 이 `lockdown,capability,landlock,yama,apparmor` 였고, `lsm=lockdown,capability,landlock,yama,apparmor,bpf` 로 GRUB 설정을 바꾸고 재부팅한 뒤 `bpf` 가 포함되어 attach가 성공하였다. **이 재부팅 요구사항 자체가 실무적 배포 비용이다** — 공유 서버 환경에서는 재부팅 가능 여부를 사전에 협의해야 하며, 이 요구사항이 SHADOW_EXEC/SHADOW_CONNECT(3.3절)에는 없다는 점이 본 논문이 두 계층을 모두 유지하는 이유다.

VM 실측 결과: `bpftool link list` 로 확인한 결과 두 혹 모두 `attach_type lsm_mac` 으로 동일하게 attach되었다. 3.4절의 공격 재현 환경으로 세 시나리오(정상 job, 알려진 바이너리 실행, 블록리스트 우회)를 재현하여 기능을 검증하였다.

```
● ● ● kshield_vpatch_lsm - live log
```

```
[정상 job] echo benign-job → 로그 없음, 오탐 없이 정상 처리
```

```
[블록리스트 우회] bash -c 'exec 3</dev/tcp/1.1.1.1/80; ...' →
```

```
LSM_CONNECT_BLOCK 사전 차단! proc=bash parent=sh dst=1.1.1.1:80 ⇒ connect() -EPERM
(mock 서버 측: "bash: connect: Operation not permitted")
```

실측 중 발견한 실제 버그: 알려진 바이너리(curl) 실행 테스트에서 처음에는 `LSM_EXEC_BLOCK` 로그가 찍혔음에도, 같은 PID가 바로 이어서 `comm=curl` 로 `connect()` 를 시도하는 `LSM_CONNECT_BLOCK` 이벤트가 발생하는 모순이 관측되었다 — `execve()` 가 실제로는 막히지 않고 성공했다는 뜻이다. `bpf_printk` 로 커널 트레이스(`trace_pipe`)를 직접 확인한 결과, 원인은 다음과 같았다: `bprm_check_security` 혹은 `execve("/usr/bin/curl", ...)` 를 `-EPERM` 으로 거부하자, 셸(dash)의 `$PATH` 탐색이 0.1밀리초 뒤 **같은 파일을 가리키는 다른 경로 문자열인 `/bin/curl` 로 재시도**하였고, 이 경로는 `suspicious_bins[]` 에 없어 통과되었다(우분투는 `/bin` 이 `/usr/bin` 의 심볼릭 링크라 실제로는 동일 파일이지만, `execve()` 에 넘어가는 문자열 자체는 다르다). 공교롭게도 `nc` 는 이미 `/bin/nc` , `/usr/bin/nc` 두 경로가 모두 등록되어 있었으나 `curl` / `wget` 는 `/usr/bin/` 경로만 등록되어 있었던 단순 누락이었다. 두 경로를 모두 등록하여 수정하였으며, 이 배열은 v3(SHADOW_EXEC, 3.3절)와 동일하므로 같은 누락이 이미 검증된 v3 결과에도 존재했었다 — 재검증하여 함께 수정하였다.

수정 전 이 버그가 존재하던 상태에서도 실제로 데이터가 외부로 나가지는 못했다 — curl이 `/bin/curl` 로 우회 실행에 성공한 직후 시도한 `connect()` 가 `security_socket_connect` 계층에서 다시 막혔기 때문이다. 이는 defense-in-depth(3.3절에서 이미 서술한 설계 근거)가 실제로 작동하는 모습을 실측 중 우연히 관찰한 사례다. 수정 후에는 두 경로 모두 exec 시점에 차단되어 CONNECT 계층까지 갈 필요 자체가 없어졌다(mock 서버 측 로그도 curl 자신의 에러 대신 `/bin/sh: 1: curl: Operation not permitted` 로 바뀌어, exec 자체가 실패했음을 확인하였다).

검 증 상 태

빌드·attach(`lsm=...,bpf` 설정 후) 성공, `execve()` / `connect()` 사전 차단 동작(수정 후), 정상 job 오탐 없음을 VM에서 확인하였다. 성능 오버헤드도 4.4절에서 v3(SHADOW_EXEC+CONNECT) 대비 측정하여 유의미한 차이가 없음을 확인하였다(처리량 p=0.6228, 지연시간 p=0.9348). 다만 탐지 동작 확인은 여전히 단일 실행 기반 정성

적 확인이며, N회 반복 탐지율, IPv6/DNS 터널링 등 SHADOW_CONNECT와 공유하는 잔여 한계(3.3절)에 대한 검증은 아직 수행하지 않았다(5장 향후 연구 참고).

3.6 v4/v5 재검토: 오탐 전제 재검토와 단계적 배포

v4 — SHADOW_EXEC의 오탐 전제 재검토. v2/v3까지 `suspicious_bins[]` 는 curl/wget 실행 자체를 목적지와 무관하게 즉시 이상 행위로 간주했다. 그러나 실제 AI 서버 워크로드에서는 워커가 모델 가중치·데이터셋을 HuggingFace/S3/사내 레지스트리 등에서 curl/wget으로 내려받는 것이 정상적인 운영이다 — "curl을 실행했는가"는 이상 행위의 신호가 아니고, "curl이 어디로 연결하는가"가 실제 신호다. 이를 반영해 curl/wget을 `suspicious_bins[]` 에서 제외하고 판단을 목적지를 아는 SHADOW_CONNECT(및 LSM `socket_connect`)에게 전적으로 넘겼다. exec 시점에는 통과시키고, 실제 `connect()` 시도 시점에 목적지가 신뢰되지 않으면 그때 잡는다 — 새 메커니즘 없이 기존 defense-in-depth 구조(exec 계층/connect 계층)를 재활용한 것이다. `nc` / `ncat` 은 AI 워커 계보에서 합법적으로 실행될 이유가 사실상 없고(리버스/바인드 셸 목적이 대부분이며, 바인드 셸은 `connect()` 를 직접 호출하지 않아 SHADOW_CONNECT만으로는 못 잡을 수 있다) 목적지와 무관하게 exec 즉시 차단을 유지한다.

v5 — audit-only(감사 전용) 모드. ModSecurity, Cloudflare WAF 등 가상 패치의 원조 도구들은 신규 룰을 배포할 때 보통 "탐지만 하고 차단은 안 함"(alert-only/dry-run) 단계를 거쳐 오탐을 관찰한 뒤에야 실제 차단으로 전환한다(2.2절). 본 논문의 기존 구현은 탐지=즉시 차단만 있었으므로, 이 단계적 배포를 지원하는 `enforce_mode` 전역 변수를 추가하였다. `watched_parents[]` 등과 달리 `const` 가 아닌 일반 전역 변수(`.data` 섹션)로 선언하여, 재컴파일 없이 유저스페이스 로더 실행 시점(`--audit-only` 플래그)에 값을 설정할 수 있다 — `enforce_mode=1`(기본)이면 기존과 동일하게 즉시 SIGKILL/-EPERM 하고, 0이면 이벤트만 기록하고 실제 차단은 건너뛴다.

VM 실측 결과: 신뢰 목적지 다운로드 시나리오(`curl http://127.0.0.1:8000/model.bin`, 더미 파일 서버)와 신뢰 안 된 목적지 시나리오(`curl http://1.1.1.1/`)를 v3(kprobe/SIGKILL)와 `kshield_vpatch_lsm` 양쪽에서, enforce/audit-only 두 모드로 재현하였다.

● ● ● v4/v5 검증 - live log

[v3, enforce] 신뢰 목적지 다운로드 → 로그 없음, 파일 정상 수신(HTTP 200)

[v3, enforce] `curl http://1.1.1.1/` →

SHADOW_CONNECT 탐지! parent=sh → proc=curl(pid=820008) dst=1.1.1.1:80 ⇒ SIGKILL 전송

[v3, audit-only] `curl http://1.1.1.1/` →

SHADOW_CONNECT 탐지! parent=sh → proc=curl(pid=820018) dst=1.1.1.1:80 ⇒ [AUDIT-ONLY] 차

```
[v3_lsm, enforce] curl http://1.1.1.1/ →
LSM_CONNECT_BLOCK 탐지! proc=curl(pid=820034) parent=sh dst=1.1.1.1:80 ⇒ connect() -EPEER
[v3_lsm, audit-only] curl http://1.1.1.1/ →
LSM_CONNECT_BLOCK 탐지! proc=curl(pid=820040) parent=sh dst=1.1.1.1:80 ⇒ [AUDIT-ONLY] cc
```

두 시나리오 모두에서 SHADOW_EXEC/LSM_EXEC_BLOCK은 더 이상 curl에 발동하지 않았고(v4가 의도한 대로), 신뢰 목적지 다운로드는 완전히 정상 처리(오탐 제거 확인)되었으며, 신뢰 안 된 목적지로의 시도는 exec 대신 connect 계층에서 여전히 정확히 차단됨을 확인하였다(탐지 능력 손실 없음). enforce/audit-only 전환도 재컴파일 없이 `--audit-only` 플래그만으로 즉시 반영됨을 확인하였다.

남는 한 계

`watched_parents[]`, `trusted_dst_ipv4[]` 등 나머지 를 배열은 여전히 rodata 컴파일 타임 상수라 재컴파일이 필요하다(5장 향후 연구). v4/v5는 각각 "실행 파일 이름 하나만으로 판단하지 않는다"와 "적어도 하나의 운영 파라미터는 재컴파일 없이 조정 가능하다"는 원칙을 부분적으로 적용한 것이며, 전체 룰셋의 런타임 갱신은 아직 미해결이다.

3.7 v6 — 감시 대상 프로세스 자신의 직접 행위 탐지

발견 경위: 제출 전 자체 검토 과정에서, `current_is_watched()` 의 판정 조건 — (a) 이미 `ai_worker_lineage` 맵에 있거나, (b) 직속 부모의 comm이 `watched_parents[]` 와 일치 — 이 둘 다 "자손 프로세스"만 커버한다는 점을 재검토하였다. `trace_lineage_fork` 는 포크가 발생할 때 "포크하는 쪽(부모)"의 comm이 `watched_parents[]` 와 일치해야 그 자식을 계보에 등록하는데, 정작 감시 대상 프로세스 본인(예: `raylet`)은 어떤 fork 이벤트에서도 "등록되는 자식" 위치에 놓이지 않으므로 계보 맵에 절대 들어가지 않는다. 그 결과 `raylet` 자신이 **fork 없이 직접** `execve()` 나 `connect()` 를 호출하면, 계보에도 없고 (자손이 아니므로) 직속 부모 검사도 통과하지 못해(raylet의 부모는 보통 systemd 등이지 raylet 자신이 아니므로) 어느 혹도 이를 감지하지 못한다. 지금까지의 모든 실험은 mock 서버가 entrypoint를 항상 `subprocess.run(["/bin/sh", "-c", ...])` 로 실행했기 때문에, 이 경로는 한 번도 노출되지 않았었다.

수정: `watched_parents[]` 와 별도로 `watched_self[]` (`raylet`, `ray::IDLE` 만 포함)를 추가하고, `current_is_watched()` 에 "내 자신의 comm이 `watched_self[]` 와 일치하는가"를 세 번째 조건으로 추가하였다. `watched_parents[]` 를 그대로 재사용하지 않은 이유는, 그 배열에는 본 PoC 편의를 위한 "python3" 처럼 매우 범용적인 이름이 포함되어 있어 자기 자신 검사에 그대로 쓰면 이 프로그램과 무관한 다른 python3 프로세스(공유 서버의 다른 사용자 스크립트 등)까지 AI 워커로 오인될 위험이 있기 때문이다. `watched_self[]` 는 실제 Ray 워커 프로세스명만 담아 이 위험을 배제한다.

VM 실측 결과: `/bin/nc` 를 파일명만 `raylet` 로 복사한 뒤(`cp /bin/nc /tmp/raylet`) 일반 bash 셸에서 직접(`/tmp/raylet 1.1.1.1 80` , fork 없이, 부모는 그냥 bash) 실행하여 재현하였다.

```

● ● ● v6 검증 - live log

[v3] /tmp/raylet 1.1.1.1 80 (수정 전이라면 무탐지) →
    SHADOW_CONNECT 탐지! parent=bash → proc=raylet(pid=820134) dst=1.1.1.1:80 ⇒ SIGKILL 전
[v3_lsm] /tmp/raylet 1.1.1.1 80 →
    LSM_CONNECT_BLOCK 탐지! proc=raylet(pid=820181) parent=bash dst=1.1.1.1:80 ⇒ connect() -
[회귀 확인] 이름을 바꾸지 않은 평범한 nc(nc 1.1.1.1 80, v3/v3_lsm 양쪽) → 로그 없음, 정상 동작(

```

이름만 `raylet` 인 프로세스가 fork 한 번 없이 자신의 직접 `connect()` 로 잡혔고, 이름을 바꾸지 않은 평범한 `nc` 는 v3·LSM 양쪽에서 여전히 아무 영향을 받지 않음을 확인하여 새 조건이 무관한 프로세스에 대한 오탐을 유발하지 않음을 검증하였다.

남는 한 계

이 수정은 `watched_self[]` 에 등록된 이름과 정확히 일치하는 comm에만 적용된다. 공격자가 프로세스 이름을 감시 목록에 없는 임의의 이름으로 바뀌서 실행하거나, 애초에 감시 대상 프로세스의 메모리 공간 안에서 코드 인젝션으로 동작하는 경우(예: 별도 프로세스/exec 없이 기존 워커의 실행 흐름 자체를 탈취)에 대해서는 여전히 다루지 않는다.

3.8 v7 — 데몬 재시작·최초 기동 시 계보 유실

발견 경위: 기획 관점 재검토 과정에서, `ai_worker_lineage` 맵이 오직 `sched_process_fork` 혹은 발동하는 순간에만 채워진다는 점을 다시 살펴보았다 — 즉 "이 데몬이 켜져 있는 동안 새로 fork되는" 프로세스만 계보에 편입된다. 그런데 실제 운영에서는 이 데몬이 이미 AI 워커 클러스터가 한참 돌아가고 있는 상태에 나중에 배포되거나, 크래시나 업데이트로 재시작되는 경우가 흔하다. 이런 경우 데몬이 뜨기 전부터 이미 존재하던 워커의 자손 프로세스들은, v6에서 다룬 "감시 대상 자신"과는 또 다른 이유로 계보에서 누락된다 — 이들은 이미 fork가 끝난 상태이므로 fork 혹은 발동할 기회가 아예 없었고, 이후 그 프로세스가 다시 fork하기 전까지는 어느 혹도 이들을 계보로 인식하지 못한다.

수정: 커널 BPF 프로그램(맵 스키마, 혹은 로직)은 전혀 건드리지 않고, 유저스페이스 로더가 attach 직후 1회 `/proc` 을 스캔하여 이미 실행 중인 프로세스들의 조상(ancestor) 체인을 직접 확인하고, 감시 대상과 일치하면 동일한 `ai_worker_lineage` 맵에 `bpf_map_update_elem` 으로 직접 채워 넣는다. 판정 기준 문자열

(`watched_parents[]` / `watched_self[]`)은 유저스페이스 코드에 별도로 옮겨 적지 않고 이미 로드된 BPF 오브젝트의 `rodata`(`skel→rodata`)에서 그대로 읽어, 커널 쪽 목록과 유저스페이스 쪽 목록이 조용히 어긋나는 문제를 원천적으로 배제하였다.

VM 실효 결과: 데몬을 켜기 전에 미리 `raylet` (파일명만 바꾼 `bash`) → `bash` → (20초 뒤) `curl` 로 이어지는 2단계 자손 체인을 백그라운드로 띄워 둔 뒤, 그 상태에서 데몬을 새로 기동하여 백필 로그와 이후 탐지 여부를 확인하였다. `curl`의 직속 부모는 `raylet` 이 아니라中间的인 `bash` 이므로, 직속 부모 검사(v2~v6 기존 로직)만으로는 절대 잡을 수 없고 오직 계보 맵(백필)에 의존해야만 잡히는 경우다.

```

● ● ● v7 검증 - live log

$ sudo ./kshield_vpatch
계보 백필: 데몬 기동 전부터 이미 실행 중이던 AI 워커 계보 프로세스 4개를 편입함
...
[04:46:17] SHADOW_CONNECT 탐지! parent=bash → proc=curl(pid=820412) dst=1.1.1.1:80 ⇒ SIGKILL

$ sudo ./kshield_vpatch_lsm
계보 백필: 데몬 기동 전부터 이미 실행 중이던 AI 워커 계보 프로세스 N개를 편입함
[04:49:22] LSM_CONNECT_BLOCK 탐지! proc=curl(pid=820428) parent=bash dst=1.1.1.1:80 ⇒ conn

```

`curl`의 직속 부모가 `bash` 인데도(v2의 직속 부모 검사로는 설명되지 않는데도) 정확히 잡혔다는 것은, 데몬 기동 전에 이미 떠 있던 `bash` 가 백필로 계보에 편입되었고 그 `bash` 가 이후 fork한 `curl` 자식이 (attach되어 있던) 라이브 `trace_lineage_fork` 훅을 통해 계보를 자동으로 물려받았다는 뜻이다. v3(SIGKILL)·LSM(사전 차단) 양쪽에서 동일하게 확인하였다.

남는 한 계

`/proc` 스캔(1차 목록 수집)과 맵 갱신(2차 판정) 사이에는 이론적으로 아주 좁은 경합 창이 있다 — 그 사이에 어떤 프로세스가 종료되고 그 PID가 무관한 새 프로세스에 재사용되면, 그 새 프로세스가 잘못 계보로 편입될 수 있다. 다만 이는 기존에도 존재하던 PID 재사용 리스크(3.3절, `sched_process_exit` 훅으로 완화)와 같은 계열이며, 그 프로세스가 종료되면 동일한 `exit` 훅으로 정리된다. 또한 이 백필은 데몬 기동 시점에 1회만 수행되므로, 데몬이 오래 켜져 있는 동안 새로 등장하는 감시 대상은 백필이 아니라 기존의 라이브 fork 훅으로 계속 정상적으로 추적된다.

4 실험

4.1 실험 환경

항목	사양
OS	Ubuntu 22.04 LTS
커널	Linux 6.8.0-138-generic
eBPF 프레임워크	libbpf + libbpf-bootstrap
Mock Ray Jobs API	Python 3 <code>http.server</code> 기반
벤치마크 도구	자체 제작 Python 스크립트(<code>benchmark_vpatch.py</code>), 요청 500회/런

4.2 실험 설계

공격 탐지 실험:

- 1. kShield-VirtualPatch 비활성 상태에서 `exploit_shadowray.py` 로 악성 `entrypoint( curl ... | sh )` 제출 → mock 서버가 이를 실행하는지 확인 (기준선)
- 2. kShield-VirtualPatch 활성 상태에서 동일 공격 재현 → SIGKILL 발동까지의 시간, 로그 출력 확인
- 3. 오탐(false positive) 측정: mock 서버가 정상적인 job(예: 단순 Python 스크립트 실행)을 처리하는 동안 kShield-VirtualPatch가 오작동하지 않는지 확인

성능 오버헤드 실험 (kShield 선행 연구와 동일한 방법론 적용):

- kShield-VirtualPatch 비활성/활성 상태에서 mock 서버의 정상 job 처리 처리량·지연시간을 N회 반복 측정
- Welch's t-test로 통계적 유의성 검증

4.3 공격 탐지 결과

구분	비활성	활성
악성 <code>entrypoint(curl sh)</code> 실행 여부	실행됨 (mock 서버가 인증 없이 그대로 실행하므로 차단 수단 없음)	curl exec 시점에 즉시 SIGKILL로 차단

다단계 체인 (python3→sh→curl) 탐지	해당 없음	탐지됨 — curl의 직속 부모는 sh이지만, fork 기반 계보 추적으로 정확히 포착
정상 job(<code>echo</code> , <code>python3 -c ...</code>) 처리	정상 처리	오탐 없이 정상 처리 (SHADOW_EXEC 미발생)

실제 kShield-VirtualPatch 로그:

```

● ● ● kshield_vpatch - live log

[00:03:55] SHADOW_EXEC 탐지! parent=sh(pid=55945) → child=curl(pid=55946) exec=/usr/bin/cu

```

trace 버퍼 분석 결과, 실행 체인은 다음과 같이 진행되었다: `python3(워커) → fork → sh` (계보 편입) → `sh` exec (감시 대상이지만 `suspicious_bins[]` 에 없어 통과) → `sh → fork → curl` (계보 편입, 2단계 추적 성공) → `curl` exec (계보 소속 + `suspicious_bins[]` 일치 → SIGKILL). 파이프의 두 번째 `sh` (`curl ... | sh`)도 동일하게 계보에 편입되었으나 `suspicious_bins[]` 에 없어 오탐 없이 통과하였다.

SHADOW_CONNECT(v3) 우회 시나리오 재현 결과: SHADOW_EXEC(v2)이 실행 파일 이름 블록리스트에 의존한다는 한계를 검증하기 위해, `suspicious_bins[]` 에 없는 방식으로 공격을 재현하였다. bash의 내장 TCP 리다이렉션 기능(`bash -c 'exec 3<>/dev/tcp/1.1.1.1/80; echo leaked >&3'`)은 curl/nc 같은 별도 바이너리를 실행하지 않으므로 SHADOW_EXEC로는 탐지되지 않는다. SHADOW_CONNECT는 이 시도를 `tcp_v4_connect` 진입 시점에 정확히 포착하였다.

```

● ● ● kshield_vpatch - live log

[13:36:02] SHADOW_CONNECT 탐지! parent=sh → proc=bash(pid=118590) dst=1.1.1.1:80 ⇒ SIGKILL

```

동일 조건에서 정상 job(`echo benign-job`)은 SHADOW_EXEC·SHADOW_CONNECT 어느 쪽도 발생시키지 않고 정상 처리되어, 새 탐지 계층 추가가 기존 정상 경로에 오탐을 유발하지 않음을 확인하였다. 개발 과정에서 `parent_comm` 필드가 로그에 빈 값으로 찍히는 버그(공통 헬퍼 함수에 배열 대신 포인터를 넘겨 `BPF_CORE_READ_STR_INT0` 의 목적지 크기 추론이 1바이트로 축소된 문제)를 실측으로 발견하여 수정하였다.

한 계

위 결과들은 단일 실행 기반 정성적 확인이며, 반복 실행에 따른 탐지율 통계(예: N회 중 탐지 성공 횟수)는 측정하지 않았다. 또한 SIGKILL까지의 정밀 지연시간(마이크로초 단위)은 별도 계측(`bpf_ktime_get_ns()` 기반 타임스탬프)이 필요하며 본 실험에서는 초 단위 로그로만 확인하였다. SHADOW_CONNECT의 나머지 한계(기존 연결 재사용, 허용 포트 위 터널링, 네트워크 불필요 공격)는 3.3절에 기술하였다.

4.4 성능 오버헤드 결과

계산 도구 자체의 정확성 검증: 아래 모든 p-value는 `attack/stat_analysis.py`가 scipy 없이 직접 구현한 정규화 불완전 베타함수 기반 t-분포 CDF로 계산된다. 제출 전 자체 검토 과정에서

`scipy.stats.ttest_ind(equal_var=False)`와 65개 케이스(논문 수치와 유사한 케이스, 소표본, 이분산, 무작위 생성 케이스 포함)로 대조 검증한 결과, 실제 버그를 하나 발견하였다 — 연속분수 계산이 `x`가 1에 가까운 영역(`t`값이 0에 매우 가까울 때, 즉 "차이가 없다"는 판정이 나와야 할 바로 그 상황)에서 수렴하지 않아, 한 케이스에서 scipy가 $p=0.996$ 을 낸 것을 이 코드는 $p=0.640$ 으로 잘못 계산하였다. 표준적인 대칭 항등식($I_x(a,b) = 1 - I_{1-x}(b,a)$)으로 항상 연속분수가 잘 수렴하는 영역에서만 계산하도록 수정하였고, 수정 후에는 65개 케이스 전체에서 scipy와 최대 오차 $1e-12$ (부동소수점 잡음 수준) 이내로 일치함을 확인하였다. 아래 표4~9에 이미 발표되었던 p-value 18개를 각 그룹의 (N, 평균, 표준편차)로부터 버그 수정 전/후 두 버전으로 재계산해 대조한 결과, 실제 측정된 `t`값들은 버그가 발동하는 극단적인 영역($t \approx 0.006$ 수준)에 전혀 해당하지 않아 기존에 발표한 값들은 영향받지 않았음을 확인하였다 — 즉 계산 도구의 결함은 실재했지만, 이미 보고된 결과의 재계산은 필요 없었다.

측정 설계: 이전 측정(SHADOW_EXEC 단독 vs 비활성 두 그룹)은 SHADOW_CONNECT와 LSM 계층을 포함하지 않는다는 한계가 명확했다. 이를 보완하여 같은 세션·같은 시스템 상태에서 세 그룹을 비교하였다 — (1)

`off`: kShield-VirtualPatch 전혀 실행 안 함(baseline), (2) `v3`:

`kshield_vpatch` (SHADOW_EXEC+SHADOW_CONNECT) 실행, (3) `v3_lsm`: `v3`에 더해

`kshield_vpatch_lsm`까지 동시 실행. 각 그룹에서 mock 서버에 정상 job을 500회/런 × 10회 반복 제출하여 측정하였다.

[표 4] 비교군별 성능 측정 결과 (N=10, 평균±표준편차)

구분	처리량 (req/s)	지연시간 (ms)
off (비활성)	1904.02 ± 6.10	0.484 ± 0.002
v3 (SHADOW_EXEC+CONNECT)	1896.63 ± 11.10	0.486 ± 0.003

v3_lsm (+LSM 사전 차단)	1898.86 ± 8.69	0.486 ± 0.002
---------------------	----------------	---------------

[표 5] Welch's t-test 결과				
비교	처 리 량 t	처 리 량 p	지 연 t	지 연 p
off vs v3	+1.845	0.0864 ns	-1.566	0.1400 ns
v3 vs v3_lsm	-0.501	0.6228 ns	+0.083	0.9348 ns
off vs v3_lsm	+1.536	0.1440 ns	-1.784	0.0932 ns

기준: ns = 유의하지 않음 (p≥0.05)

세 그룹 간 처리량·지연시간 차이는 어느 쌍을 비교해도 통계적으로 유의미하지 않았다. 이로써 이전 측정의 공백 — SHADOW_CONNECT를 포함한 재측정, LSM 사전 차단 계층의 오버헤드 측정 — 을 모두 메웠다. 단, p>0.05는 "차이가 없음"을 증명하는 것이 아니라 "차이를 기각하지 못한 것"임을 유의해야 한다. 완전한 동등성 주장을 위해서는 TOST 기반 동등성 검정과 더 큰 표본(N≥30)이 필요하며, 이는 향후 연구 과제로 남긴다.

한 계 1 - 절대 수 치 의 세 셴 의 존 성

표 4의 절대 처리량(~1900 req/s)은 이전 측정(586 req/s대)과 크게 다르다. 세 그룹 모두 같은 세션·재부팅 이후 상태에서 측정하였으므로 그룹 간 상대 비교는 유효하지만, 절대 수치는 측정 시점의 시스템 배경 부하에 크게 좌우되는 것으로 보인다. 벤치마크 결과는 세션 간에 직접 비교해서는 안 되며, 반드시 동일 세션 내 상대 비교로 해석해야 함을 보여주는 사례다.

무관한 프로세스에 대한 영향 측정: tcp_v4_connect / tcp_v6_connect kprobe와 security_socket_connect LSM 혹은 AI 워커 계보 여부와 무관하게 시스템의 모든 프로세스의 모든 connect() 호출에서 실행된다 — current_is_watched() (계보 map 조회 + 부모 comm 비교)가 목적지를 읽기도 전에 먼저 실행되고, 계보에 속하지 않으면 그 자리에서 바로 반환하기 때문이다. 표 4는 mock 서버 (AI 워커 계보 자신)의 처리량·지연시간만 다루므로, 이 "무관한 프로세스" 경로의 오버헤드는 attack/benchmark_unrelated_connect.py 로 별도 측정하였다 — AI 워커 계보 밖의 프로세스(직속 부모가 쉐이지 watched_parents[] 와 일치하지 않아 계보에 편입되지 않음)가 로컬 더미 TCP 리스너에 반복 connect() 하는 지연시간을 동일한 off/v3/v3_lsm 3그룹으로 측정하였다.

[표 6] 무관한 프로세스의 connect() 성능 (N=10, 평균±표준편차)				
--	--	--	--	--

구분	처 리 량 (req/s)	지 연 시 간 (ms)
off_unrelated	2557.70 ± 295.07	0.352 ± 0.051
v3_unrelated	2546.47 ± 206.83	0.350 ± 0.039
v3_lsm_unrelated	2600.01 ± 22.16	0.341 ± 0.003

[표 7] Welch's t-test 결과 (무관한 프로세스)

비교	처 리 량 t	처 리 량 p	지 연 t	지 연 p
off vs v3	+0.099	0.9227 ns	+0.083	0.9347 ns
v3 vs v3_lsm	-0.814	0.4362 ns	+0.773	0.4590 ns
off vs v3_lsm	-0.452	0.6617 ns	+0.693	0.5059 ns

기준: ns = 유의하지 않음 (p≥0.05)

AI 워커 계보와 무관한 connect()에서도 세 그룹 간 유의미한 처리량·지연시간 차이는 관측되지 않았다. 다만 off_unrelated/v3_unrelated의 표준편차(처리량 기준 각각 295, 207)가 v3_lsm_unrelated(22)보다 눈에 띄게 커서 그룹 간 비교가 정확히 대칭적이지는 않다 — N=10의 소표본에서 흔한 배경 부하 변동으로 보이며, eBPF/LSM 활성화가 오히려 변동성을 줄인다고 해석할 근거는 없다.

한 계 2 - 무관한 프로세스 측정도 특정 시나리오로 국한됨

위 측정은 (1) 루프백(127.0.0.1) 목적지, (2) 요청 간 동시성 없이 순차적으로 1개씩 connect()하는 워크로드만 다룬다. `current_is_watched()` 가 목적지 확인 전에 반환되므로 루프백 여부 자체는 결과에 영향이 없지만, 다수의 무관한 프로세스가 동시다발적으로 connect()를 시도하는(고동시성) 상황에서의 영향은 측정하지 않았으며 향후 연구 과제로 남긴다.

fork 집약적 워크로드 측정: 위 측정들의 job은 fork 단계가 얇아(1~2단계) 계보 추적(매 fork·exit마다 BPF map 갱신)의 오버헤드를 충분히 드러내지 못했을 수 있다는 우려가 있었다. `benchmark_vpatch.py --fork-heavy` 로 job 하나당 `/bin/true` 를 50회 반복 fork+exec하는 entryptpoint를 사용하여 동일한 3그룹을 재측정하였다.

[표 8] fork 집약적 워크로드 성능 측정 결과 (N=10, 평균±표준편차)		
구분	처 리 량 (req/s)	지 연 시 간 (ms)
off_fork	605.56 ± 1.95	1.582 ± 0.005
v3_fork	603.12 ± 3.12	1.588 ± 0.008
v3_lsm_fork	601.51 ± 1.94	1.593 ± 0.005

[표 9] Welch's t-test 결과 (fork 집약적 워크로드)				
비교	처 리 량 t	처 리 량 p	지 연 t	지 연 p
off vs v3	+2.098	0.0531 ns	-2.235	0.0409 *
v3 vs v3_lsm	+1.382	0.1870 ns	-1.463	0.1644 ns
off vs v3_lsm	+4.648	0.0002 ***	-5.009	0.0001 ***

기준: ns = 유의하지 않음($p \geq 0.05$), * $p < 0.05$, *** $p < 0.001$

여기서는 처음으로 통계적으로 유의미한 차이가 관측되었다. off 대비 v3_lsm는 처리량 -0.67% (605.56→601.51 req/s), 지연시간 +0.7%(1.582→1.593ms)로 그 자체는 작지만, 표준편차가 매우 작아 (± 2 수준) N=10에서도 강한 유의성($p < 0.001$)으로 검출되었다. v3 단독 대비 v3_lsm 추가분은 유의하지 않아($p = 0.16 \sim 0.19$), LSM 계층 자체가 오버헤드를 크게 늘리기보다는 v3(SHADOW_EXEC+CONNECT)의 계보 추적 비용이 fork 빈도가 높아지며 누적되어 통계적으로 드러난 것으로 해석된다. 이는 앞선 "유의미한 차이 없음" 결과와 모순되지 않는다 — **오버헤드가 워크로드의 fork 빈도에 비례**하며, fork가 드문 워크로드에서는 노이즈에 묻혀 보이지 않다가 fork가 빈번해지면 드러난다는, 더 정확한 그림을 보여준다. 대략적으로(end-to-end HTTP 지연시간 기준의 거친 추정치이며 개별 fork/exit 혹은 격리 측정한 것은 아님) job당 약 50회 늘어난 fork+exit 쌍에 대해 11마이크로초의 추가 지연이 관측되어, 추가 fork+exit 쌍 하나당 약 0.2마이크로초 수준의 비용으로 추산된다. 정확한 per-fork 비용은 `bpf_ktime_get_ns()` 기반 격리 측정이 필요하며 향후 연구로 남긴다.

다중비교 보정과 통계적 검정력에 대한 유의: 본 절까지 표5/7/9에서 총 18회의 Welch's t-test를 수행하였다. 다중비교를 보정하지 않은 $\alpha = 0.05$ 를 그대로 쓰면 우연히 유의한 결과가 나올 확률이 누적되므로, Bonferroni 보정($\alpha = 0.05/18 \approx 0.0028$)을 적용해 재검토하였다 — off vs v3_lsm(처리량 $p = 0.0002$, 지연시간 $p = 0.0001$)

은 보정 후에도 유의성이 유지되지만, 표9의 off vs v3 지연시간($p=0.0409$)은 보정 기준을 통과하지 못해 유의하지 않은 것으로 재해석해야 한다. 즉 이번 실험에서 다중비교 보정을 통과하는 유일한 유의미한 차이는 off vs v3_lsm(fork 집약적 워크로드) 하나이며, 이는 본문에서 이미 핵심 발견으로 강조한 바로 그 결과다. 아울러 $N=10$ 은 검정력이 낮아, 위의 "유의하지 않음(ns)" 판정들은 "차이가 없음의 증명"이 아니라 "이 표본 크기로는 작은 차이를 검출하지 못했을 수 있다"는 의미로 읽어야 한다 — 실제로 fork가 얇은 조건에서 ns였던 것과 동일한 종류의 오버헤드가 fork가 빈번해지자 유의하게 검출된 사실 자체가, 검정력 부족이 실재하는 우려임을 뒷받침한다. 엄밀한 동등성 주장에는 TOST와 $N \geq 30$ 이 필요하다는 점은 이미 향후 연구로 명시하였다.

5 결론 및 향후 연구

본 논문은 AI 서빙 프레임워크의 알려진 RCE 취약점(ShadowRay)에 대해, 코드 수정이나 재배포 없이 eBPF로 가상 패치를 적용하는 방법을 제시하였다. SHADOW_EXEC과 SHADOW_CONNECT 두 계층은 반복 횟수가 아닌 단일 이상 행위 발생을 기준으로 하므로, 기존 kShield⁸의 빈도 기반 탐지가 가진 구조적 한계(공격자가 한 번의 접근으로 데이터를 탈취하는 경우 탐지 불가)를 보완한다.

VM 실측 결과, python3(워커)→sh→curl로 이어지는 2단계 공격 체인은 curl 실행 시점에 (SHADOW_EXEC), bash `/dev/tcp/` 내장 기능을 이용한 우회 시도는 연결 시도 시점에 (SHADOW_CONNECT) 각각 정확히 차단하였으며, 두 경우 모두 정상 job은 오탐 없이 통과함을 확인하였다. 비활성/SHADOW_EXEC+CONNECT/LSM 사전 차단을 포함한 3개 그룹을 10회씩 반복 측정한 결과, fork 단계가 얇은 워크로드와 무관한 프로세스의 connect() 성능 모두에서 통계적으로 유의미한 차이는 관측되지 않았다(off vs v3: 처리량 $p=0.0864$, 지연시간 $p=0.1400$; v3 vs v3+LSM: 처리량 $p=0.6228$, 지연시간 $p=0.9348$; Welch's t-test). 다만 job당 fork를 50배가량 늘린 워크로드에서는 처음으로 통계적으로 유의미한 차이가 나타났다(off vs v3+LSM: 처리량 $p=0.0002$, 지연시간 $p=0.0001$) — 절대 크기는 작지만(처리량 -0.67% , 지연시간 $+0.7\%$), 오버헤드가 워크로드의 fork 빈도에 비례함을 보여준다.

개발 과정에서 여섯 차례의 설계 반복이 있었다. 첫째, "직속 부모만 확인"하는 초기 설계(v1)가 다단계 공격 체인을 놓치고 정상 job도 오탐하는 문제를 실측으로 발견하여 프로세스 계보(lineage) 추적 방식(v2)으로 재설계하였다. 둘째, v2의 실행 파일 이름 블록리스트(`suspicious_bins[]`)가 bash 내장 기능 같은 우회에 취약함을 확인하여, 연결 시도 자체를 감시하는 SHADOW_CONNECT(v3)를 추가하였다. 셋째, 비동기 SIGKILL의 잔여 한계(3.3절)를 해소하기 위해 LSM 혹 기반 동기적 사전 차단(3.5절)을 추가로 구현·검증하는 과정에서, `suspicious_bins[]`가 `/usr/bin/curl`은 막으면서 셸의 `$PATH` 재탐색이 시도하는 동일 파일의 다른 경로 문자열(`/bin/curl`)은 놓치는 버그를 발견하였다 — 이는 이미 "검증됨"으로 문서화했던 v3에도 동일하게 존재하던 누락이었다. 이 경험들은 행위 기반 탐지 메커니즘을 설계할 때 (1) 실행 체인의 깊이, (2) 감시 신호가

"특정 도구의 사용"이 아니라 "행위의 본질적 불변량"에 기반해야 함, 그리고 (3) 경로 기반 블록리스트는 같은 대상을 가리키는 모든 문자열 별칭(alias)을 빠짐없이 열거해야 함을 반드시 고려해야 함을 보여준다. 다만 SHADOW_CONNECT조차도 완전한 차단은 아니며, 이미 열려 있는 연결의 재사용, 허용된 프로토콜 위의 터널링, 네트워크가 불필요한 로컬 전용 공격에는 대응하지 못한다 — 이는 특정 구현의 결함이라기보다 행위 기반 탐지 일반의 본질적 한계에 가깝다.

넷째, SHADOW_EXEC이 "특정 도구의 사용" 자체를 이상 행위로 간주한다는 (2)의 원칙을 curl/wget에는 일관되게 적용하지 못했음을 뒤늦게 재검토하였다(3.6절, v4) — curl/wget은 모델·데이터셋 다운로드에도 쓰이는 이중 용도 도구라, 목적지와 무관한 즉시 차단은 정당한 사용까지 오탐으로 죽인다. 판단을 목적지 인지 계층(SHADOW_CONNECT)에 넘기도록 재설계하고 VM에서 신뢰/비신뢰 목적지 양쪽을 재검증하였다. 아울러 신규 룰을 먼저 감사(audit-only)로 배포한 뒤 오탐을 관찰하고 차단으로 전환하는, WAF 가상 패치의 표준 운영 방식(2.2절)을 지원하는 `enforce_mode` 런타임 토글을 추가하였다(v5).

다섯째, 제출 전 자체 검토 과정에서 계보 판정 로직이 "감시 대상 프로세스의 자손"만 커버하고 감시 대상 프로세스 자신의 직접 행위는 구조적으로 놓친다는 공백을 발견하였다(3.7절, v6) — 지금까지의 모든 실험이 mock 서버의 서브프로세스 실행 방식 때문에 이 경로를 한 번도 노출하지 않았던, 실험 설계 자체의 사각지대였다. `watched_self[]` 라는 별도의 좁은 목록으로 자기 자신의 comm도 확인하도록 수정하고 VM에서 검증하였다. 이 발견은 (2)의 원칙("특정 도구의 사용"이 아닌 "행위의 본질적 불변량"에 기반)이 계보 판정 로직 자체에도 적용되어야 함을 보여준다 — "누구의 자손인가"뿐 아니라 "나 자신이 감시 대상인가"까지 함께 물어야 완전하다.

여섯째, 실험이 아닌 기획 관점에서 배포 시나리오를 재검토하는 과정에서, `ai_worker_lineage` 맵이 fork 후에 의해서만 채워진다는 설계가 "데몬이 클러스터보다 나중에 뜬다"는 상황을 전혀 고려하지 않았음을 발견하였다(3.8절, v7) — 데몬이 이미 실행 중인 워커 뒤에 나중에 붙거나(최초 기동) 재시작되면, 그 시점 이전에 fork가 끝난 자손 프로세스들은 스스로 다시 fork하기 전까지 계보에서 완전히 누락된다. 커널 BPF 프로그램은 그대로 두고, 유저스페이스 로더가 기동 시 `/proc` 을 스캔해 이미 떠 있는 프로세스들의 계보를 동일한 맵에 직접 채워 넣는 방식으로 해소하였으며, 직속 부모 검사만으로는 절대 잡을 수 없는 손자뻘(2단계) 프로세스 체인으로 VM에서 재현·검증하였다. 이 발견은 탐지 로직의 정확성뿐 아니라 "데몬의 생애주기(기동·재시작 시점)가 클러스터의 생애주기와 어떻게 어긋날 수 있는가"라는, 실험만으로는 드러나지 않는 배포 설계 차원의 질문이 별도로 필요함을 보여준다.

향후 연구 과제는 다음과 같다. 본 논문의 핵심 주장(탐지 가능, 오탐 없음, 성능 무영향) 자체의 신뢰도에 직결되는 항목과, 적용 범위를 넓히는 지엽적인 항목을 구분하여 제시한다.

핵심 주장의 검증 강화 (우선순위 높음)

- **실제 Ray 클러스터 환경 검증:** 현재는 mock 서버 기반 재현이며, 실제 Ray 설치 환경에서의 검증이 필요하다. mock 서버는 entrypoint를 항상 서브프로세스로 실행하므로, 실제 Ray의 job 실행 경로(멀티프로세싱, actor 기반 실행 등)가 이와 다르다면 3.7절(v6)에서 다룬 것과 유사한 계보 추적 사각지대가 다른 형태로 남아있을 가능성을 배제할 수 없다.
- **오탐 시나리오 확장 검증:** `echo`, `python3 -c` 같은 단순 job과 curl/wget 정상 다운로드(3.6절)는 실측으로 검증하였다. 다만 AI 워커가 그 밖의 서브프로세스를 실행하거나 외부와 통신하는 더 복잡한 합법적 워크로드(예: 데이터 전처리 파이프라인, pip/conda 패키지 설치, 정상적인 API 호출)와의 충돌 가능성은 아직 다양한 시나리오로 검증하지 못했다.
- **탐지 반복성 및 정밀 지연시간 측정:** 공격 탐지는 단일 실행 기반 정성적 확인에 그쳤다. N회 반복 탐지율과 `bpf_ktime_get_ns()` 기반 정밀 SIGKILL 지연시간 측정이 필요하다. `kshield_vpatch_lsm` (3.5절)도 동일하게 N회 반복 탐지율 측정이 남아 있다.
- **동등성 검정(TOST) 수행:** 4.4절에서 다중비교 보정과 검정력 한계를 논의하였듯, 성능 무영향을 통계적으로 엄밀히 증명하려면 TOST 기반 동등성 검정과 표본 크기 확대($N \geq 30$)가 필요하다.

적용 범위 확장 (우선순위 낮음)

- **다른 CVE로의 일반화:** 현재는 ShadowRay 한 사례에 특화되어 있다. Triton, TorchServe, MLflow 등 다른 프레임워크의 알려진 CVE에 대해서도 유사한 방식의 룰을 구축하고 공통 프레임워크로 일반화할 필요가 있다.
- **런타임 룰 설정 지원:** `watched_parents[]`, `suspicious_bins[]`, `trusted_dst_ipv4[]` 가 컴파일 타임에 고정되어 있어, 새로운 CVE 대응이나 신뢰 목적지 추가 시마다 재컴파일 필요하다 (`enforce_mode` 만 예외적으로 재컴파일 없이 런타임 전환 가능, 3.6절). BPF map 기반 런타임 룰 갱신 메커니즘 도입이 필요하다.
- **고동시성 환경에서의 시스템 전역 오버헤드:** AI 워커 계보와 무관한 프로세스의 순차적 connect() 성능에는 유의미한 영향이 없음을 확인하였다(4.4절, `benchmark_unrelated_connect.py`). 다만 이는 순차 요청 기준이며, 다수의 무관한 프로세스가 동시다발적으로 connect()를 시도하는 고동시성 상황에서의 영향은 아직 측정하지 않았다.
- **정밀 per-fork 비용 측정:** fork 집약적 워크로드에서 통계적으로 유의미한 오버헤드(4.4절, job당 약 50회 늘어난 fork당 약 0.2마이크로초 추정)를 확인하였으나, 이는 end-to-end HTTP 지연시간 기준의 거친 추정치다. `bpf_ktime_get_ns()` 로 혹 자체의 실행 시간만 격리 측정해 정확한 per-fork 비용을 검증해야 한다.

- **실제 Ray의 fork 패턴 재현:** 본 실험의 fork 집약적 워크로드(`/bin/true` 반복 실행)는 실제 Ray 태스크의 워커 프로세스 생성 패턴을 정밀하게 재현한 것은 아니다. 실제 Ray 클러스터(위 항목 참고)의 태스크 처리량·fork 빈도를 프로파일링하여 더 현실적인 워크로드로 재측정할 필요가 있다.
- **IPv6 신뢰 목적지 목록 지원:** 현재 IPv6는 loopback(`::1`) 외 모든 목적지를 의심으로 간주하며, IPv4의 `trusted_dst_ipv4[]` 에 대응하는 신뢰 목록이 없다.
- **DNS 터널링 등 잔여 우회 대응:** 허용된 프로토콜(DNS 등) 위로 데이터를 은닉하는 터널링, 기존 정상 연결에 얹혀가는 방식 등 SHADOW_CONNECT로도 막지 못하는 우회에 대한 추가 탐지 계층 연구가 필요하다.
- **Falco/Tetragon과의 정량적 비교:** 기존 범용 런타임 보안 도구 5.6에 동일한 룰을 이식했을 때와의 성능·정확도 비교가 필요하다.
- **프로세스명 위장 및 인프로세스 코드 실행 대응:** `watched_self[]` (3.7절)는 comm이 정확히 일치하는 경우만 다룬다. 공격자가 감시 목록에 없는 이름으로 바이너리를 배치하거나, 별도 프로세스 생성 없이 기존 감시 대상 프로세스의 메모리 공간 안에서 코드 실행 흐름 자체를 탈취하는 경우(예: 인터프리터 내부의 코드 인젝션)는 다루지 않는다.

§ 참고문헌

- [1] Oligo Security, "ShadowRay: First Known Attack Campaign Targeting AI Workloads Exploited In The Wild," Mar. 2024. oligo.security/blog/shadowray-attack-ai-workloads-actively-exploited-in-the-wild
- [2] Anyscale, "Update on Ray CVEs CVE-2023-6019, CVE-2023-6020, CVE-2023-6021, CVE-2023-48022, CVE-2023-48023," Anyscale Blog, 2024. anyscale.com/blog/update-on-ray-cves-...
- [3] MITRE/NVD, "CVE-2023-48022," National Vulnerability Database, 2023. nvd.nist.gov/vuln/detail/CVE-2023-48022
- [4] GitHub Advisory Database, "Ray has arbitrary code execution via jobs submission API (GHSA-6wgj-66m2-xxp2)," 2023. github.com/advisories/GHSA-6wgj-66m2-xxp2
- [5] Falco, "Container Runtime Security," The Falco Authors, CNCF, 2024. falco.org
- [6] Isovalent, "Tetragon: eBPF-based Security Observability and Runtime Enforcement," GitHub, 2023. github.com/cilium/tetragon

- [7] OWASP Foundation, "Virtual Patching Best Practices," 2024. owasp.org/www-community/Virtual_Patching_Best_Practices
- [8] 이상헌, "kShield: eBPF 기반 AI 모델 보안 프레임워크 구현 코드," GitHub, 2026. github.com/sangheon-lee1028/Ai-ebpf
- [9] 이상헌, "kShield-VirtualPatch: eBPF 기반 AI 서빙 프레임워크 취약점 가상 패치 구현 코드," GitHub, 2026. github.com/sangheon-lee1028/kShield-VirtualPatch

kShield-VirtualPatch · paper_draft.md

GitHub 저장소의 원문과 동기화됨